ErrorBuddy

How a beginner-friendly compiler is built, and how it works

C++17 · standard library only · 5 phases · ~1,100 lines

```
$ ./errorbuddy missing_semicolon.txt

Error on line 2:
Expected ';' at the end of the statement.
Suggestion: Add a semicolon after '10'.

(Found during: Syntax Analysis)
```

Compiler errors talk like grammars

TYPICAL COMPILER (gcc on the equivalent C)

```
1  int a = 5;
2  int b = 10
3  printf("%d", a + b);

program.c:3:5: error:
expected ',' or ';' before 'printf'
```

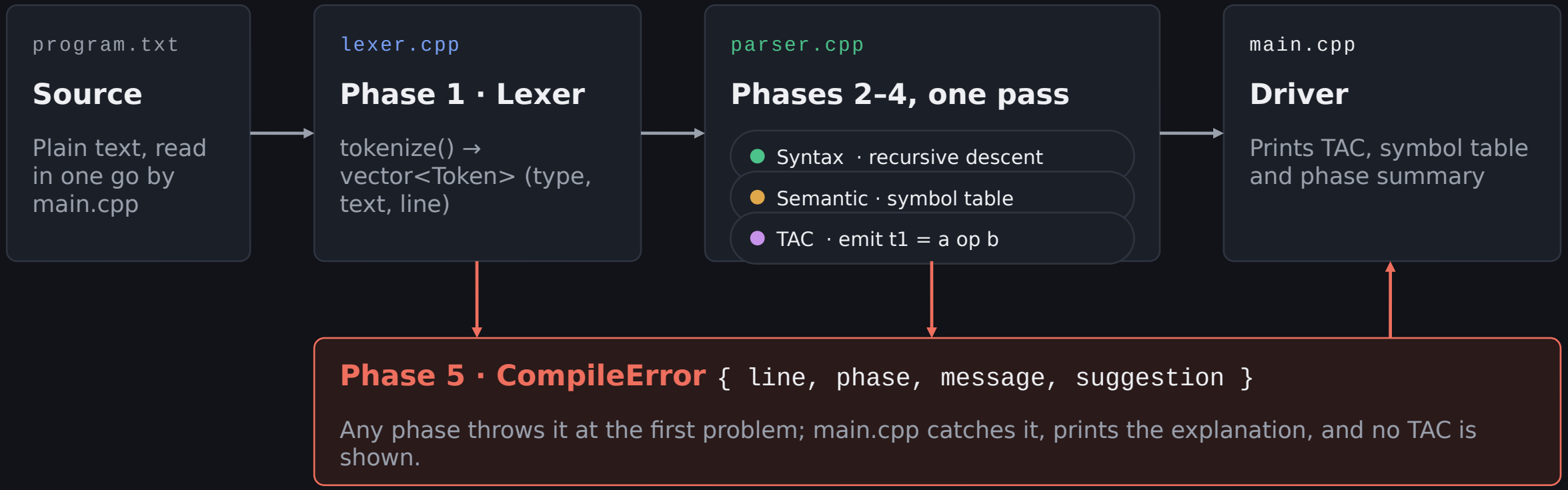
- Wrong line: points after the mistake
- Grammar words: "expected … before"

ERRORBUDDY

```
Error on line 2:
Expected ';' at the end of the
statement.
Suggestion: Add a semicolon
after '10'.
(Found during: Syntax Analysis)
```

- Line where the fix belongs
- Plain English + one concrete fix

Architecture: five phases in three source files



- web/server.py runs the same binary → JSON → playground
- Exit codes: 0 ok · 1 compile error · 2 bad usage

Phase 1 · The lexer makes tokens

INPUT

```
int c = a + b * 2;
```

OUTPUT: vector<Token>

int	c	=	a	+	b	*	2	;
KEYWORD	IDENTIFIER	ASSIGN	IDENTIFIER	OPERATOR	IDENTIFIER	OPERATOR	NUMBER	SEMICOLON

Skips & tracks

- Spaces and // comments
- Counts newlines, so every token carries its line

Rejects

- Unknown symbols such as @
- Numbers too large for an int
- Names that start with a digit: 2nd

Hands on

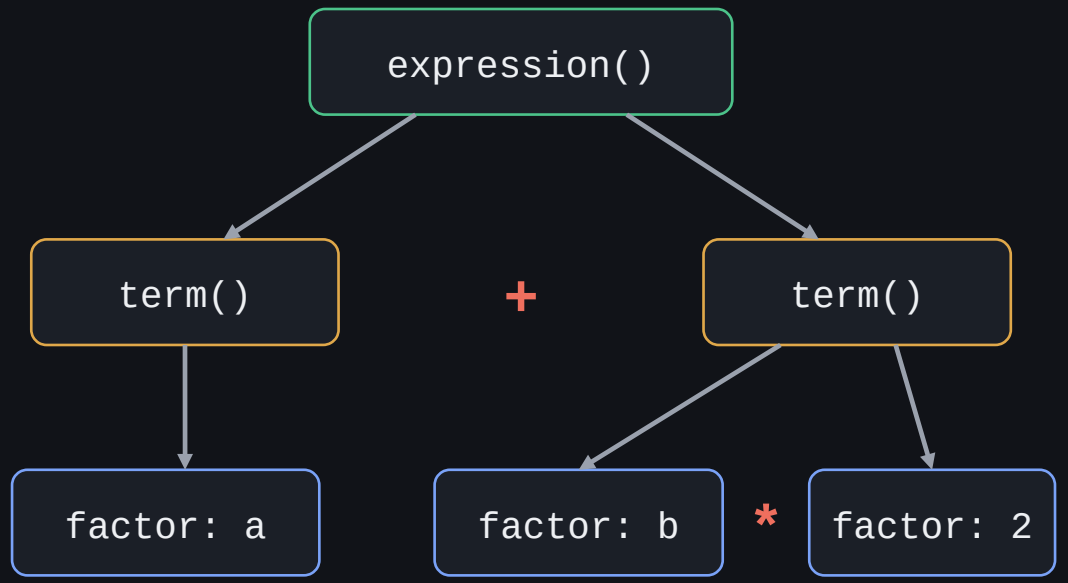
- Token list ending in END_OF_FILE
- --tokens prints it as a table

Phase 2 · One function per grammar rule

```
program     → statement*
statement   → declaration
              | assignment | printStmt
declaration → "int" ID "=" expr ";"
assignment  → ID "=" expr ";"
printStmt   → "print" expr ";"
expression  → term ((+|-) term)*
term        → factor ((*|/) factor)*
factor      → NUM | ID | ( expr )
```

Precedence comes from the grammar, not a table. while loops in expression() and term() make operators left-associative: 10 - 4 - 3 = (10 - 4) - 3.

CALL TREE FOR a + b * 2



b * 2 is finished inside the right-hand term() before + is applied, so the TAC computes it first.

Phase 3 · The symbol table checks meaning

`std::map<string, Symbol>` AFTER `examples/valid.txt`

name	declared	value
age	line 2	20
score	line 3	50
bonus	line 5	9

There is no input and no loop, so every value is known while compiling. That is how a / zero is caught even when zero is a variable.

Undeclared or misspelled

`scroe = 2;` → Did you mean 'score'?

Declared twice

'age' already declared on line 1

Used in its own declaration

`int a = a + 1;`

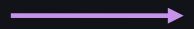
Division by zero, overflow

`a / zero`, `8 / (4 - 4)`, `int range`

Phase 4 · Three-address code

SOURCE

```
int age = 20;
int score = 10;
score = score + age * 2;
int bonus = (score - 5) / 5;
print score;
print bonus * (age + 1);
```



TAC

```
age = 20          t4 = t3 / 5
score = 10        bonus = t4
t1 = age * 2      print score
t2 = score + t1   t5 = age + 1
score = t2        t6 = bonus * t5
t3 = score - 5    print t6
```

- At most one operator per instruction
- Temps t1, t2… skip names already used
- Printed only if the whole program passes

Phase 5 · Every error is a structured object

line	phase	message	suggestion
2	Syntax Analysis	Expected ';' at the end of the statement.	Add a semicolon after '10'.

Mistake	ErrorBuddy says
`x + * 3`	Two operators in a row: '+' followed by '*'.
`(x + 3`	Missing ')' to close the '(' opened on line 2.
`int a = -5;`	Negative numbers such as '-5' are not supported directly. Write 0 - 5.
`prnt x;`	Unknown word 'prnt'. Did you mean the keyword 'print'?
`int a;`	Variable 'a' is declared without a starting value.

Diagnosis reads the surrounding tokens

expectSemicolon(): the statement is over but the next token is not ';'. What is it instead?

`)`	Found an extra ')' that has no matching '('.
`=`	Unexpected '='. A statement can contain only one '='.
`x 5 (`	Same line → Missing operator between the two values.
`other`	Expected ';' — reported on the line of the previous token.

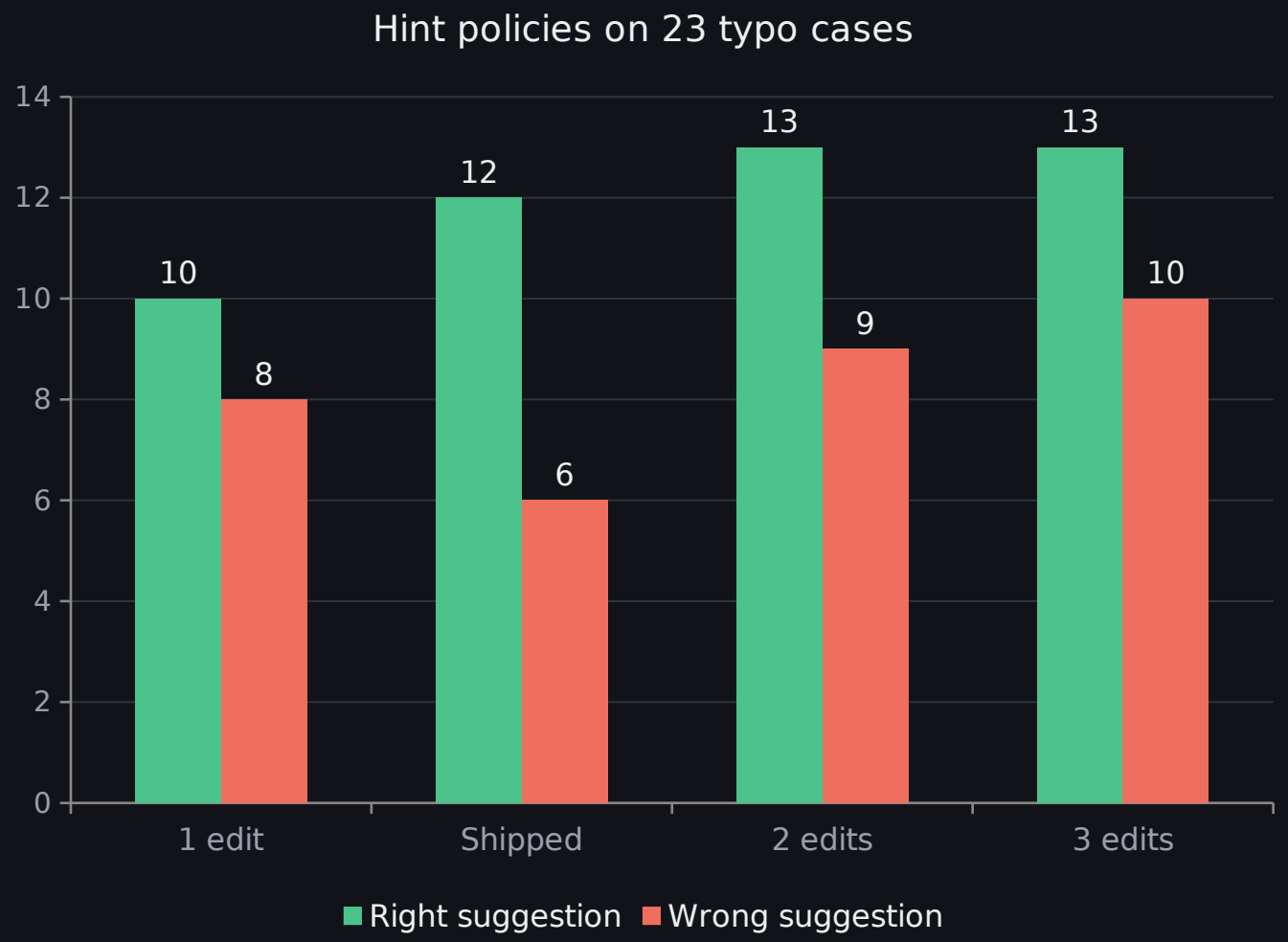
The last rule is why a missing ';' on line 2 is reported on line 2, not on the line where the parser noticed it.

Typo hints use edit distance

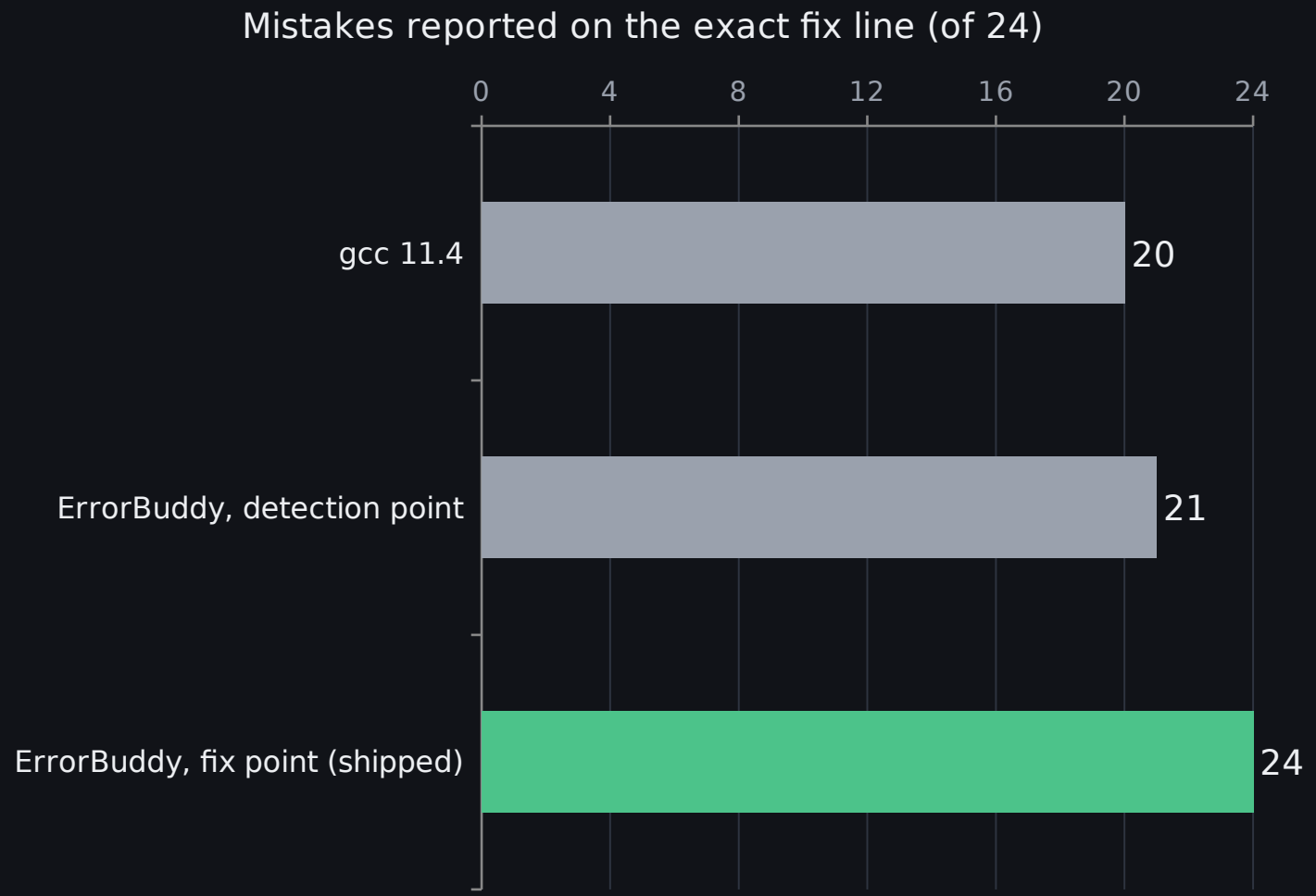
```
scroe → score   2 edits
prnt  → print   1 edit
mx    → max     1 edit
```

Shipped rule
Allow 1 edit for names of up to 3 letters, 2 for longer names, and never suggest the word itself.

Levenshtein distance in a single rolling row (parser.cpp, editDistance). Measured on 23 typo cases from case_study/corpus.py.



Case study: the line where the fix belongs



24 / 24

mistakes caught by ErrorBuddy (gcc: 23)

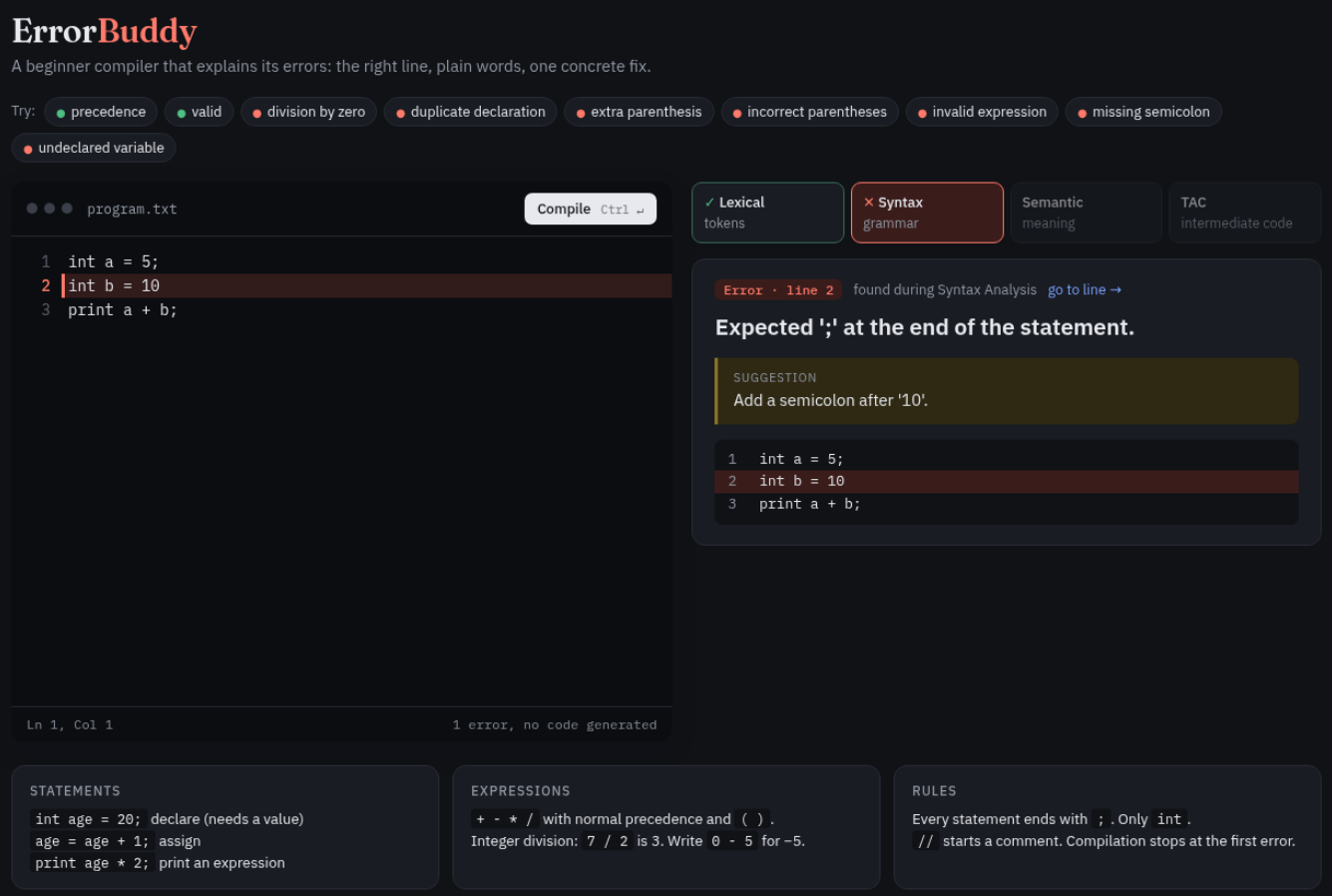
0

reports later than the fix line (gcc: 3)

3

late cases were all a missing ';'

Seeing it work: the web playground



- make web → http://localhost:8795
- web/server.py: Python standard library only
- POST /api/compile runs the real ./errorbuddy --tokens
- Parses its output into JSON: tokens, error, TAC, symbols
- Editor highlights the error line; phase strip shows where it stopped

Built and tested today, and what comes next

BUILT

- int variables with a required starting value
- + - * / with precedence and parentheses
- print any expression; // comments
- TAC, symbol table, phase summary
- Stops at the first error
- Web playground

10 / 10

example programs match their expected output in make test

NEXT (FROM THE PROPOSAL)

- if/else, while and for loops
- Comparison and logical operators, printf
- Error recovery: several errors per run
- Functions and arrays; char and float
- Constant folding and other optimisations

Kept small on purpose so every phase is complete. The right column is planned, not built.

ErrorBuddy

A complete small compiler, lexer to three-address code, whose main contribution is error messages that teach: the right line, plain words, and a concrete fix.

- make test
- make web → localhost:8795
- python3 case_study/run_study.py